

Jenkins – Task2 – (Master - Slave)

1. Configure 2 slave machines in the Jenkins master.

Objective

The objective of this task is to configure **one Jenkins Master and two Jenkins Agent EC2 instances** and establish SSH-based communication between the Master and both Agents.

The configuration uses a dedicated Linux user named **jenkins** on all three machines instead of using **root** or the default **ec2-user**

Purpose

In a Jenkins distributed architecture, the **Jenkins Master/Controller** manages Jenkins while the **Agents** perform the actual build and execution work.

Pre Requirements:

Launch 3 Ec2 Server

JAVA

Install Jenkins with (<https://www.jenkins.io/>)

Commands:

Memory: `Free -m`

For CPU: `cat /proc/cpuinfo`

For OS Config Details: `cat /etc/os-release`

Steps:

1. Create Dedicated jenkins User on Master

Create the Jenkins user: `sudo useradd -m -s /bin/bash Jenkins`

Verify: `id Jenkins`

Check home directory: `/home/Jenkins`

2. Create Dedicated jenkins User on Agent-01/ Agent-02

Create the dedicated user: `sudo useradd -m -s /bin/bash Jenkins`

Verify: `sudo ls -ld /home/Jenkins`

3.Install Java on Agent-01/ Agent-02

`sudo dnf install java-21-amazon-corretto -y`

Verify: `java -version`

Then locate Java: `readlink -f $(which java)`

4.Prepare SSH Directory on Agent-01/ Agent-02

`sudo mkdir -p /home/jenkins/.ssh`

Set ownership: `sudo chown jenkins:jenkins /home/jenkins/.ssh`

Set permissions: `sudo chmod 700 /home/jenkins/.ssh`

Verify: `sudo ls -ld /home/jenkins/.ssh`

5.Create SSH Key on Jenkins Master

Switch to the dedicated Jenkins user: `sudo su – Jenkins`

Verify: `whoami`

Create the SSH directory: `mkdir -p ~/.ssh`

Set permission: `chmod 700 ~/.ssh`

Now generate the SSH key: `ssh-keygen -t rsa -b 4096 -m PEM -f ~/.ssh/jenkins_agent_key`

This creates:

Jenkins – Task2 – (Master - Slave)

`/home/jenkins/.ssh/jenkins_agent_key`

`/home/jenkins/.ssh/jenkins_agent_key.pub`

Verify SSH Keys: `ls -la ~/.ssh`

Check private key:

→ `ls -l ~/.ssh/jenkins_agent_key.pub`

→ `ls -l ~/.ssh/jenkins_agent_key`

Run:

`chmod 600 ~/.ssh/jenkins_agent_key`

`chmod 644 ~/.ssh/jenkins_agent_key.pub`

Get the Public Key : `cat ~/.ssh/jenkins_agent_key.pub`

6.Add Public Key to Agent-01/ Agent-02

Create the authorized keys file: `sudo touch /home/jenkins/.ssh/authorized_keys`

Now edit it: `sudo vi /home/jenkins/.ssh/authorized_keys`

Then:

→ `sudo chown jenkins:jenkins /home/jenkins/.ssh/authorized_keys`

Set: `sudo chmod 600 /home/jenkins/.ssh/authorized_keys`

Verify: `sudo ls -l /home/jenkins/.ssh/authorized_keys`

7.Verify SSH Configuration on Agent-01/ Agent-02

`sudo sshd -T | grep -E 'pubkeyauthentication|authorizedkeysfile'`

You should see public-key authentication enabled.

Also check SSH service:

→ `sudo systemctl status sshd`

→ `sudo systemctl restart sshd`

8.Test Master → Agent-01/ Agent-01 SSH

Make sure you are: Jenkins User

SSH: `ssh -o IdentitiesOnly=yes -i ~/.ssh/jenkins_agent_key jenkins@Pub.ip`

9.Add SSH Private Key to Jenkins Credentials

Open: Jenkins → Manage Jenkins → Credentials

Create: → Kind: → SSH Username with private key

Use: → ID: → jenkins-agent-ssh

Username: jenkins

Private Key: Select → Enter directly

On the Jenkins Master, display the private key:

`sudo su - jenkins`

`cat ~/.ssh/jenkins_agent_key`

Copy the entire private key into Jenkins Credentials.

Very important

Do not put: → jenkins_agent_key.pub → into Jenkins.

Jenkins needs: → jenkins_agent_key → the private key.

10.Create Agent-01/ Agent-02 Node

Go to: Manage Jenkins → Nodes → New Node

Node name: jenkins-agent-01

Select: Permanent Agent

Jenkins – Task2 – (Master - Slave)

Click: [Create](#)

Configure:

Remote root directory: [/home/Jenkins](#)

This is important because our Linux user is: **Jenkins**

and its home directory is: `/home/Jenkins`

Number of executors: 2

Labels: agent-01/ agent-02

Usage

Select: Only build jobs with label expressions matching this node

This ensures jobs explicitly requesting: `agent-01/ agent-02`

11.Configure Agent-01/ Agent-02 SSH Launcher

Launch method: Launch agents via SSH

Host: Worker/slave server ip

Credentials→Select: **jenkins-agent-ssh**

Implementation:

```
Last login: Wed Sep 30 06:06:37 UTC 2026 from 103.160.27.229
[ec2-user@ip-172-31-19-208 ~]$ whoami
ec2-user
[ec2-user@ip-172-31-19-208 ~]$ sudo su - jenkins
Last login: Wed Sep 30 06:16:18 UTC 2026 on pts/3
[jenkins@ip-172-31-19-208 ~]$ whoami
jenkins
[jenkins@ip-172-31-19-208 ~]$ ssh -o IdentitiesOnly=yes -i ~/.ssh/jenkins_agent_key jenkins@34.235.117.10

^C
[jenkins@ip-172-31-19-208 ~]$ ssh -o IdentitiesOnly=yes -i ~/.ssh/jenkins_agent_key jenkins@34.235.117.105
The authenticity of host '34.235.117.105 (34.235.117.105)' can't be established.
ED25519 key fingerprint is SHA256:UqJUicEdBGIsHmv/h+LZn5x2fEHZae86xVBapony3vc.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '34.235.117.105' (ED25519) to the list of known hosts.
```

```
#
# Amazon Linux 2023
https://aws.amazon.com/linux/amazon-linux-2023
```

```
[jenkins@ip-172-31-24-29 ~]$
logout
Connection to 34.235.117.105 closed.
[jenkins@ip-172-31-19-208 ~]$ ssh -o IdentitiesOnly=yes -i ~/.ssh/jenkins_agent_key jenkins@13.220.188.203
The authenticity of host '13.220.188.203 (13.220.188.203)' can't be established.
ED25519 key fingerprint is SHA256:7umgyQpeQuwWBWUSSdFortKEHH94/I+5wJpBzpu2xQk.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '13.220.188.203' (ED25519) to the list of known hosts.
```

```
#
# Amazon Linux 2023
https://aws.amazon.com/linux/amazon-linux-2023
```

```
[jenkins@ip-172-31-25-174 ~]$
```

Jenkins / Manage Jenkins / Nodes

Nodes

Add, remove, control and monitor the various nodes that Jenkins runs jobs on.

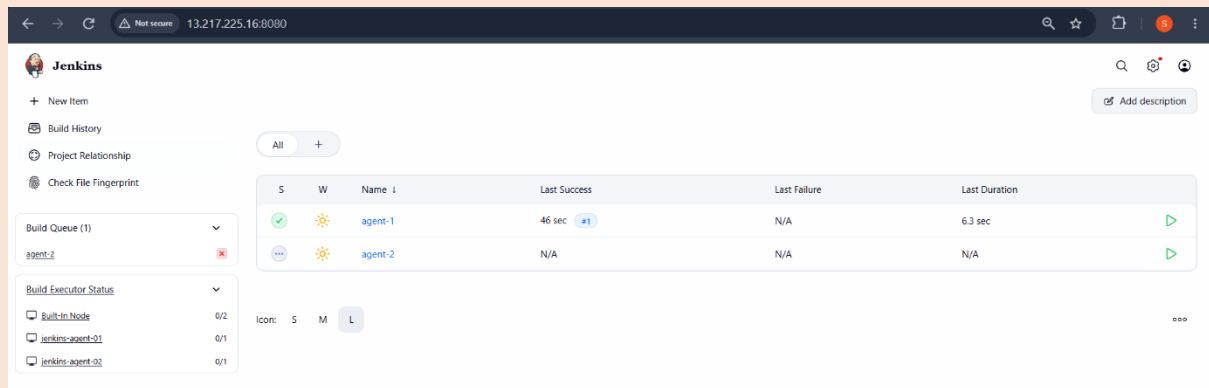
[+ New Node](#) [Configure Monitors](#) [Refresh](#)

S	Name	Architecture	Clock Difference	Free Disk Space	Free Swap Space	Free Temp Space	Response Time
	Built-In Node	Linux (amd64)	In sync	5.70 GiB	0 B	451.84 MiB	0ms
	jenkins-agent-01	Linux (amd64)	In sync	5.99 GiB	0 B	456.43 MiB	52ms
	jenkins-agent-02		N/A	N/A	N/A	N/A	N/A
last checked		1 ms	1 ms	1 ms	1 min 13 sec	1 min 13 sec	1 min 13 sec

Icon: S M L

Legend

Jenkins – Task2 – (Master - Slave)



Conclusion

The Jenkins Master and two Jenkins Agents were successfully designed using an SSH-based distributed Jenkins architecture.

A dedicated Linux user named jenkins was created on all three EC2 instances. Java 21 was installed on both Agents, and SSH public-key authentication was configured between the Master and the Agents.

2. Configure webhooks to the Jenkins job.

Objective

To configure a GitHub webhook that automatically notifies Jenkins when a change is pushed to a GitHub repository, allowing the configured Jenkins job to respond to the GitHub event and initiate the required SCM polling/build process.

Purpose

A GitHub webhook provides an event-driven mechanism for connecting GitHub with Jenkins. Instead of relying only on Jenkins repeatedly checking GitHub for changes, GitHub can send an HTTP POST notification to the Jenkins webhook endpoint when a repository event occurs.

Implemented Clear Steps

- Step 1: Verified that Jenkins was running on port 8080.
- Step 2: Opened the Jenkins job configuration.
- Step 3: Configured Git under Source Code Management.
- Step 4: Added the GitHub repository URL.
- Step 5: Opened the Build Triggers section.
- Step 6: Enabled GitHub hook trigger for GITScm polling.
- Step 7: Saved the Jenkins job configuration.
- Step 8: Determined the Jenkins webhook endpoint: http://Pub_ip:8080/github-webhook/
- Step 9: Opened the GitHub repository.
- Step 10 : Navigated to **Settings → Webhooks**.
- Step 11: Selected **Add webhook**.
- Step 12: Entered the Jenkins webhook URL.
- Step 13 : Selected application/json as the payload content type.
- Step 14: Configured the webhook to listen for the **push** event.
- Step 15: Enabled the webhook.
- Step 16: Verified the webhook delivery from GitHub.

Jenkins – Task2 – (Master - Slave)

Step 17: Created a test change in the Git repository.

Step 16: Committed the change.

Step 17: Pushed the change to GitHub.

Step 18: Verified that GitHub generated a push webhook delivery.

Step 19: Verified the webhook response.

Step 20: Checked Jenkins for the resulting build.

Step 21: Opened Jenkins Console Output.

Step 22: Verified that Jenkins retrieved the updated Git repository content.

Step 23: Confirmed the GitHub-to-Jenkins automation workflow.

Implementation:

The image shows two screenshots of the Jenkins web interface. The top screenshot displays the 'webhook_job' configuration page. On the left, a sidebar contains links: Status, Changes, Workspace, Build Now, Delete Project, GitHub Hook Log, Configure, and Rename. The main area shows a green checkmark icon, the job name 'webhook_job', a 'webhook link', and a 'Permalinks' section with four links: 'Last build (#1), 3 min 6 sec ago', 'Last stable build (#1), 3 min 6 sec ago', 'Last successful build (#1), 3 min 6 sec ago', and 'Last completed build (#1), 3 min 6 sec ago'. Below this is a 'Builds' table with two entries: '#2 17:32' and '#1 17:29', both marked as successful with green checkmarks. A context menu is open over the '#2' build, showing options: 'Changes', 'Console Output', and 'Edit build information'. The bottom screenshot shows the 'Console Output' for build #2. The browser address bar indicates the URL '54.211.65.108:8080/job/webhook_job/2/console'. The Jenkins header shows 'Jenkins / webhook_job / #2'. The console output text reads: 'Started by user admin', 'Running as SYSTEM', 'Building on the built-in node in workspace /var/lib/jenkins/workspace/webhook_job', '[webhook_job] \$ /bin/sh -xe /tmp/jenkins4881688885027561470.sh', '+ sleep 15', and 'Finished: SUCCESS'.

Conclusion

The GitHub webhook was successfully configured between the GitHub repository and Jenkins. Jenkins was configured to respond to GitHub webhook events using GitHub hook trigger for GITScm polling. A test Git push was used to verify the integration, confirming that changes in the repository

Jenkins – Task2 – (Master - Slave)

can notify Jenkins and initiate the configured job workflow.

3. Configure Poll SCM and Build Periodically options in the Jenkins job.

Objective

To configure and understand the **Poll SCM** and **Build Periodically** options in a Jenkins job and verify the difference between these two scheduling mechanisms.

Purpose

Jenkins provides scheduling options that can automatically check or execute jobs at specified times.

For this task, two options are configured:

→ Poll SCM — Jenkins periodically checks the configured source-code repository for changes and triggers a build when a relevant SCM change is detected.

→ Build Periodically — Jenkins runs the job according to the configured schedule regardless of whether there has been a source-code change

Steps:

Part 1 — Configure Poll SCM

Step 1 — Open the Jenkins Job

→ Select your GitHub-connected Jenkins job → **Configure**

Step 2 — Verify Source Code Management

Source Code Management: Git

Verify the repository URL.

Verify the branch: Main

Step 3— Enable Poll SCM

Build Triggers: ☒ Poll SCM

Part — Save Poll SCM Configuration

Part 2 — Configure Build Periodically

→ Your Job → Configure

scroll to Build Triggers

Step 1 — Enable Build Periodically → * * * * *

Part 2 — Save the Configuration

Implementation:

Jenkins – Task2 – (Master - Slave)

 **Jenkins** / poll SCM and Build Job

Status

Changes

Workspace

Build Now

Delete Project

Git Polling Log

Configure

Rename

poll SCM and Build Job

Permalinks

- Last build (#1), 23 sec ago
- Last stable build (#1), 23 sec ago
- Last successful build (#1), 23 sec ago
- Last completed build (#1), 23 sec ago

Builds >

Filter

Today

#1 17:56

← → ↻ 🔒 Net secure 54.211.65.108:8080/job/poll%20SCM%20and%20Build%20Job/1/console

 **Jenkins** poll SCM and Build Job / #1

Status

Changes

Console Output

Delete build #1

Polling Log

Git Build Data

Edit build information

Console

Download

```
Started by an SCM change
Running as SYSTEM
Building on the built-in node in workspace /var/11b/jenkins/workspace/poll SCM and Build Job
The recommended git tool is: NONE
No credentials specified
Cloning the remote Git repository
Cloning repository https://github.com/Sunith3210/jenkins_test.git
> git init /var/11b/jenkins/workspace/poll SCM and Build Job # timeout=10
Fetching upstream changes from https://github.com/Sunith3210/jenkins_test.git
> git --version # timeout=10
> git --version # 'git version 2.50.1'
> git fetch --tags --force --progress -- https://github.com/Sunith3210/jenkins_test.git :refs/heads/*:refs/remotes/origin/* # timeout=10
> git config remote.origin.url https://github.com/Sunith3210/jenkins_test.git # timeout=10
> git config --add remote.origin.fetch :refs/heads/*:refs/remotes/origin/* # timeout=10
Avoid second fetch
> git rev-parse refs/remotes/origin/main^{commit} # timeout=10
Checking out Revision 9993a7202d32f09551bee517ad7fba30abc155 (refs/remotes/origin/main)
> git config core.sparsecheckout # timeout=10
> git checkout -f 9993a7202d32f09551bee517ad7fba30abc155 # timeout=10
Commit message: "Update README.md"
First time build. Skipping changelog.
Finished: SUCCESS
```

 **Jenkins** / poll SCM and Build Job

Status

Changes

Workspace

Build Now

Delete Project

Configure

Rename

poll SCM and Build Job

Permalinks

- Last build (#1), 1 min 40 sec ago
- Last stable build (#1), 1 min 40 sec ago
- Last successful build (#1), 1 min 40 sec ago
- Last completed build (#1), 1 min 40 sec ago

Builds >

Filter

Today

#2 17:58

#1 17:56

Jenkins – Task2 – (Master - Slave)

The screenshot shows the Jenkins configuration interface for a job named 'poll SCM and Build Job'. In the 'Triggers' section, 'Build periodically' is selected with a schedule of '*****'. A warning message states: 'Do you really mean "every minute" when you say "*****"? Perhaps you meant "H*****" to poll once per hour. Would last have run at Tuesday, September 29, 2026, 5:58:00 PM Coordinated Universal Time; would next run at Tuesday, September 29, 2026, 5:59:00 PM Coordinated Universal Time'. Below this, 'Poll SCM' is also selected. The console output shows the job running as SYSTEM, fetching changes from a remote Git repository, and successfully checking out the latest revision.

Triggers

Set up automated actions that start your build based on specific events, like code changes or scheduled times.

- ☐ Trigger builds remotely (e.g., from scripts) ?
- ☐ Build after other projects are built ?
- ☒ Build periodically ?

Schedule ?

⚠ Do you really mean "every minute" when you say "*****"? Perhaps you meant "H*****" to poll once per hour
Would last have run at Tuesday, September 29, 2026, 5:58:00 PM Coordinated Universal Time; would next run at Tuesday, September 29, 2026, 5:59:00 PM Coordinated Universal Time

☐ GitHub hook trigger for GITScm polling ?

☐ Poll SCM ?

Jenkins / poll SCM and Build Job #2

Status

Changes

Console Output

Delete build #2

Git Build Data

Edit build information

Previous Build

Console

Download

```
Started by timer
Running as SYSTEM
Building on the built-in node in workspace /var/lib/jenkins/workspace/poll SCM and Build Job
The recommended git tool is: NONE
No credentials specified
> git rev-parse --resolve-git-dir /var/lib/jenkins/workspace/poll SCM and Build Job/.git # timeout=10
Fetching changes from the remote Git repository
> git config remote.origin.url https://github.com/Sunith3210/jenkins_test.git # timeout=10
Fetching upstream changes from https://github.com/Sunith3210/jenkins_test.git
> git --version # timeout=10
> git --version # 'git version 2.50.1'
> git fetch --tags --force --progress -- https://github.com/Sunith3210/jenkins_test.git +refs/heads/*:refs/remotes/origin/* # timeout=10
> git rev-parse refs/remotes/origin/main^{commit} # timeout=10
Checking out Revision 9993a7202d32f09551bee517a6d7fba30abc155 (refs/remotes/origin/main)
> git config core.sparsecheckout # timeout=10
> git checkout -f 9993a7202d32f09551bee517a6d7fba30abc155 # timeout=10
Commit message: "Update README.md"
> git rev-list --no-walk 9993a7202d32f09551bee517a6d7fba30abc155 # timeout=10
Finished: SUCCESS
```

Conclusion

Poll SCM and Build Periodically were successfully configured in the Jenkins job.

Poll SCM allows Jenkins to periodically check the configured Git repository and initiate a build when changes are detected.

Build Periodically executes the Jenkins job according to its configured schedule without requiring a source-code change

4. Take a backup of the Jenkins server using a **bash-script**.

Objective

To create and execute a Bash script that takes a compressed backup of the Jenkins server's JENKINS_HOME directory and verifies that the backup archive was successfully created and can be read.

Purpose

Jenkins stores important configuration and operational information inside the JENKINS_HOME directory. Taking regular backups protects Jenkins configuration and allows the environment to be restored if the Jenkins server is lost or corrupted

Steps:

Verified the Jenkins home directory at **/var/lib/jenkins**.

Created the external backup directory: **/backup/Jenkins**

/usr/local/bin/jenkins_backup.sh

Jenkins – Task2 – (Master - Slave)

Conclusion

The Jenkins server was successfully backed up using a Bash script.

The script automates the complete backup process by stopping Jenkins, creating a timestamped compressed archive of JENKINS_HOME, restarting Jenkins, and validating the resulting backup.

The backup contains the important Jenkins configuration and operational data required for the next task, including jobs, plugins, node configuration, users, credentials-related configuration, and Jenkins secrets.

Take a backup of Jenkins using the **ThinBackup** plugin.

Objective

To install, configure, and use the Jenkins **ThinBackup plugin** to create a backup of Jenkins configuration and job-related data.

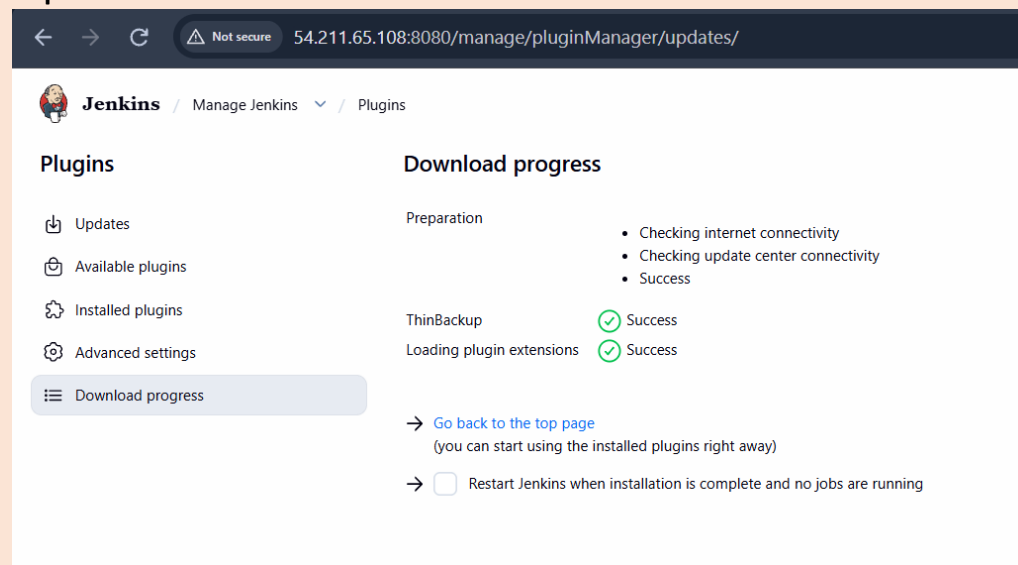
Purpose

ThinBackup provides Jenkins administrators with a convenient method of creating Jenkins backups through the Jenkins management interface.

Implemented Clear Steps

- Opened the Jenkins dashboard.
- Navigated to Manage Jenkins → Plugins.
- Searched for the ThinBackup plugin.
- Installed the ThinBackup plugin.
- Verified the plugin installation.
- Created the backup directory: /backup/thinbackup
- Changed the backup directory ownership to: jenkins:Jenkins
- Restricted access to the backup directory.
- Opened the ThinBackup configuration. Configured: /backup/thinbackup
- Saved the ThinBackup configuration.
- Initiated a manual backup using the ThinBackup interface.

Implementation:



The screenshot shows the Jenkins web interface at the URL `54.211.65.108:8080/manage/pluginManager/updates/`. The page title is "Jenkins / Manage Jenkins / Plugins". On the left, a sidebar menu lists "Updates", "Available plugins", "Installed plugins", "Advanced settings", and "Download progress" (which is selected). The main content area is titled "Download progress" and shows the status of the plugin installation. Under the "Preparation" section, there are three items: "Checking internet connectivity", "Checking update center connectivity", and "Success". Under the "ThinBackup" section, there are two items: "ThinBackup" with a green checkmark and "Success", and "Loading plugin extensions" with a green checkmark and "Success". At the bottom, there are two links: "Go back to the top page (you can start using the installed plugins right away)" and "Restart Jenkins when installation is complete and no jobs are running" (which is unchecked).

Jenkins – Task2 – (Master - Slave)

ThinBackup Configuration

Backup directory ?
/var/lib/jenkins/backup

Backup schedule for full backups ?

Backup schedule for differential backups ?

Max number of backup sets ?
-1

Files excluded from backup (regular expression) ?

☒ Wait until Jenkins is idle to perform a backup ?
Force Jenkins to quiet mode after specified minutes ?
120

☒ Backup build results ?
☐ Backup build archive ?

Save Apply

```
[jenkins@ip-172-31-28-197 jenkins]$ s
-bash: s: command not found
[jenkins@ip-172-31-28-197 jenkins]$ ls
Devops.pem
authorized_keys
backup
com.cloudbees.hudson.plugins.folder.config.AbstractFolderConfiguration.xml
config.xml
credentials.xml
fingerprints
github-plugin-configuration.xml
hudson.model.UpdateCenter.xml
hudson.plugins.build_timeout.global.GlobalTimeoutConfiguration.xml
hudson.plugins.build_timeout.operations.BuildStepOperation.xml
hudson.plugins.email_ext.ExtendedEmailPublisher.xml
hudson.plugins.git.GitSCM.xml
hudson.plugins.git.GitTool.xml
hudson.plugins.gradle.enriched.EnrichedSummaryConfig.xml
hudson.plugins.gradle.injection.InjectionConfig.xml
hudson.plugins.timestamp.TimestamperConfig.xml
hudson.tasks.Mailer.xml
hudson.tasks.Shell.xml
hudson.triggers.SCMTrigger.xml
identity.key.enc
io.jenkins.plugins.junit.storage.JunitTestResultStorageConfiguration.xml
jenkins.fingerprints.GlobalFingerprintConfiguration.xml
jenkins.install.InstallUtil.lastExecVersion
jenkins.install.UpgradeWizard.state
jenkins.model.ArtifactManagerConfiguration.xml
[jenkins@ip-172-31-28-197 jenkins]$ cd backup/
[jenkins@ip-172-31-28-197 backup]$ ll
total 16
drwxr-xr-x. 5 jenkins jenkins 16384 Sep 29 18:42 FULL-2026-09-29_18-42
[jenkins@ip-172-31-28-197 backup]$
```

Conclusion:

The ThinBackup plugin was successfully installed and configured on the Jenkins server.

A dedicated backup location was created at: /backup/thinbackup

A manual ThinBackup backup was successfully initiated and verified

Jenkins – Task2 – (Master - Slave)

- Set up a new Jenkins server and restore the backup taken in task 4.

Objective

To set up a new Jenkins controller/server and restore the Jenkins environment from the Bash backup created during Task 4. The restoration should recover the Jenkins configuration, jobs, build history, users, plugins, credentials, nodes/agents, and job trigger configurations.

Purpose

The purpose of this task is to demonstrate **Jenkins disaster recover**

Steps:

- Located the Bash backup under: /backup/jenkins/
- Checked the existing Jenkins and Java versions.
- Created a new EC2 instance for the replacement Jenkins controller
- Stopped Jenkins before restoring the backup.
- Moved the newly created /var/lib/jenkins directory to a temporary location.
- Transferred the Task 4 Bash backup to the new Jenkins server.
- Verified the transferred backup archive.
- Extracted the backup into /var/lib.
- Restored the Jenkins Home directory.

Corrected ownership using: `sudo chown -R jenkins:jenkins /var/lib/Jenkins`

Verification / Observation

Jenkins jobs

Job configuration

Build history

Plugins

Credentials

Jenkins agents

Agent labels

GitHub trigger configuration

Poll SCM configuration

Build Periodically configuration

Implementation:

```
[jenkins@ip-172-31-28-197 backup]$ cd ..
[jenkins@ip-172-31-28-197 jenkins]$ ls
Devops.pem
FULL-2026-09-29_18-42.tar
authorized_keys
backup
com.cloudbees.hudson.plugins.folder.config.AbstractFolderConfiguration.xml
config.xml
credentials.xml
fingerprints
github-plugin-configuration.xml
hudson.model.UpdateCenter.xml
hudson.plugins.build_timeout.global.GlobalTimeoutConfiguration.xml
hudson.plugins.build_timeout.operations.BuildStepOperation.xml
hudson.plugins.emailext.ExtendedEmailPublisher.xml
hudson.plugins.git.GitSCM.xml
hudson.plugins.git.GitTool.xml
hudson.plugins.gradle.enriched.EnrichedSummaryConfig.xml
hudson.plugins.gradle.injection.InjectionConfig.xml
hudson.plugins.timesamper.TimesamperConfig.xml
hudson.tasks.Mailer.xml
hudson.tasks.Shell.xml
hudson.triggers.SCMTrigger.xml
identity.key.enc
io.jenkins.plugins.junit.storage.JunitTestResultStorageConfiguration.xml
jenkins.fingerprints.GlobalFingerprintConfiguration.xml
jenkins.install.InstallUtil.lastExecVersion
jenkins.install.UpgradeWizard.state
jenkins.model.ArtifactManagerConfiguration.xml
[jenkins@ip-172-31-28-197 jenkins]$ scp -i Devops.pem FULL-2026-09-29_18-42.tar slave-1054.221.148.181:/home/slave-1
slave-1054.221.148.181's password:
FULL-2026-09-29_18-42.tar
[jenkins@ip-172-31-28-197 jenkins]$
[jenkins@ip-172-31-28-197 jenkins]$
```

```
jenkins.model.GlobalBuildDiscarderConfiguration.xml
jenkins.model.GlobalComputerRetentionCheckIntervalConfiguration.xml
jenkins.model.JenkinsLocationConfiguration.xml
jenkins.security.ResourceDomainConfiguration.xml
jenkins.tasks.filters.EnvVarsFilterGlobalConfiguration.xml
jenkins.telemetry.Correlator.xml
jobs
logs
nodeMonitors.xml
nodes
org.jenkinsci.plugins.displayurlapi.DefaultDisplayURLProviderGlobalConfiguration.xml
org.jenkinsci.plugins.github_branch_source.GitHubConfiguration.xml
org.jenkinsci.plugins.workflow.flow.GlobalDefaultFlowDurabilityLevel.xml
org.jenkinsci.plugins.workflow.libs.GlobalLibraries.xml
org.jenkinsci.plugins.workflow.libs.GlobalUntrustedLibraries.xml
org.jvnet.hudson.plugins.thinbackup.ThinBackupPluginImpl.xml
plugins
queue.xml
queue.xml.bak
secret.key
secret.key.not-so-secret
secrets
updates
userContent
users
workspace
```

```
100% 10KB 12.6MB/s 00:00
```

Jenkins – Task2 – (Master - Slave)

```
[slave-1@ip-172-31-28-201 ~]$ pwd
/home/slave-1
[slave-1@ip-172-31-28-201 ~]$
[slave-1@ip-172-31-28-201 ~]$
[slave-1@ip-172-31-28-201 ~]$
[slave-1@ip-172-31-28-201 ~]$ mkdir .ssh
[slave-1@ip-172-31-28-201 ~]$ cd .ssh/
[slave-1@ip-172-31-28-201 .ssh]$ ls
[slave-1@ip-172-31-28-201 .ssh]$ vi authorized_keys
[slave-1@ip-172-31-28-201 .ssh]$ chmod 700 authorized_keys
[slave-1@ip-172-31-28-201 .ssh]$ ls
authorized_keys
[slave-1@ip-172-31-28-201 .ssh]$ getent passwd slave-1
slave-1:x:1001:1001::/home/slave-1:/bin/bash
[slave-1@ip-172-31-28-201 .ssh]$ sudo mkdir -p /home/slave-1/.ssh
[sudo] password for slave-1:
[slave-1@ip-172-31-28-201 .ssh]$ sudo chown slave-1:slave-1 /home/slave-1/.ssh
[slave-1@ip-172-31-28-201 .ssh]$ sudo chmod 700 /home/slave-1/.ssh
[slave-1@ip-172-31-28-201 .ssh]$ sudo -u slave-1 ssh-keyscan -H 54.211.65.108 >> /home/
[slave-1@ip-172-31-28-201 .ssh]$ sudo chown slave-1:slave-1 /home/slave-1/.ssh/known_ho
sudo chmod 600 /home/slave-1/.ssh/known_hosts
[slave-1@ip-172-31-28-201 .ssh]$ cd ..
[slave-1@ip-172-31-28-201 ~]$ ls
[slave-1@ip-172-31-28-201 ~]$ pwd
/home/slave-1
[slave-1@ip-172-31-28-201 ~]$ ls
FULL-2026-09-29_18-42.tar
[slave-1@ip-172-31-28-201 ~]$
```

← → ↺ ⚠ Not secure 52.23.230.244:8080/user/admin/builds

 **Jenkins** / admin ▾ / Builds

 **admin**

Profile

Builds

My Views

Account

Appearance

Preferences

Security

Experiments

Credentials

Builds

	S	Build	Time Since	↑
Icon:	S	M	L	

Jenkins – Task2 – (Master - Slave)

```
FULL-2026-09-29_18-42/jobs/poll SCM and Build Job/builds/42/log
FULL-2026-09-29_18-42/jobs/poll SCM and Build Job/builds/42/changelog.xml
FULL-2026-09-29_18-42/jobs/poll SCM and Build Job/builds/42/build.xml
FULL-2026-09-29_18-42/jobs/poll SCM and Build Job/builds/43/
FULL-2026-09-29_18-42/jobs/poll SCM and Build Job/builds/43/log
FULL-2026-09-29_18-42/jobs/poll SCM and Build Job/builds/43/changelog.xml
FULL-2026-09-29_18-42/jobs/poll SCM and Build Job/builds/43/build.xml
FULL-2026-09-29_18-42/jobs/poll SCM and Build Job/builds/44/
FULL-2026-09-29_18-42/jobs/poll SCM and Build Job/builds/44/log
FULL-2026-09-29_18-42/jobs/poll SCM and Build Job/builds/44/changelog.xml
FULL-2026-09-29_18-42/jobs/poll SCM and Build Job/builds/44/build.xml
FULL-2026-09-29_18-42/jobs/poll SCM and Build Job/builds/45/
FULL-2026-09-29_18-42/jobs/poll SCM and Build Job/builds/45/log
FULL-2026-09-29_18-42/jobs/poll SCM and Build Job/builds/45/changelog.xml
FULL-2026-09-29_18-42/jobs/poll SCM and Build Job/builds/45/build.xml
FULL-2026-09-29_18-42/jobs/poll SCM and Build Job/builds/46/
FULL-2026-09-29_18-42/jobs/poll SCM and Build Job/builds/46/log
FULL-2026-09-29_18-42/jobs/poll SCM and Build Job/builds/46/changelog.xml
FULL-2026-09-29_18-42/jobs/poll SCM and Build Job/builds/46/build.xml
FULL-2026-09-29_18-42/jobs/poll SCM and Build Job/builds/47/
FULL-2026-09-29_18-42/jobs/poll SCM and Build Job/builds/47/log
FULL-2026-09-29_18-42/jobs/poll SCM and Build Job/builds/47/changelog.xml
FULL-2026-09-29_18-42/jobs/poll SCM and Build Job/builds/47/build.xml
FULL-2026-09-29_18-42/jobs/Backup_job/
FULL-2026-09-29_18-42/jobs/Backup_job/config.xml
FULL-2026-09-29_18-42/jobs/Backup_job/builds/
FULL-2026-09-29_18-42/jobs/Backup_job/builds/permalinks
FULL-2026-09-29_18-42/users/
FULL-2026-09-29_18-42/users/admin_555c7f4c8b658eebd5ddddd6320d7cb05888aa47db1ced6e739898145db7d0ea3/
FULL-2026-09-29_18-42/users/admin_555c7f4c8b658eebd5ddddd6320d7cb05888aa47db1ced6e739898145db7d0ea3/config.xml
FULL-2026-09-29_18-42/users/noreply_56d91afb0c76875c12c4507836b8a58581529552c0d279e7690d7cc6c2fe266e/
FULL-2026-09-29_18-42/users/noreply_56d91afb0c76875c12c4507836b8a58581529552c0d279e7690d7cc6c2fe266e/config.xml
FULL-2026-09-29_18-42/nodes/
FULL-2026-09-29_18-42/nodes/slave-1/
FULL-2026-09-29_18-42/nodes/slave-1/config.xml
FULL-2026-09-29_18-42/installedPlugins.xml
FULL-2026-09-29_18-42/backup-completed.info
[slave-2@ip-172-31-18-201 jenkins]$
```

i-02ff76e20c02556e7 (Slave-2)

PublicIPs: 52.23.230.244 PrivateIPs: 172.31.18.201

The screenshot shows the Jenkins web interface. At the top, there's a navigation bar with 'Jenkins' and a '+ New Item' button. Below that, there's a 'Build History' link. The main content area is divided into two panels. The left panel shows the 'Build Queue (1)' with a dropdown menu and a 'poll SCM and Build Job' entry. Below that, the 'Build Executor Status' panel shows 'Built-in Node' (offline) and 'slave-1' (offline). The right panel shows a table of builds.

S	W	Name ↓	Last Success	Last Failure	Last Duration
...	☀	Backup_job	N/A	N/A	N/A
✓	☀	poll SCM and Build Job	1 min 12 sec #49	N/A	0.13 sec
✓	☀	webhook_job	2 hr 43 min #2	N/A	15 sec

At the bottom, there's a 'Icon: S M L' selector.

Conclusion

The Jenkins disaster-recovery procedure was successfully demonstrated by creating a new Jenkins server and restoring the Bash backup created.